Python Programming

Important Viva Questions & Answers

Complete Reference Guide

Unit 1: Core Python

Q1. Why are Python strings immutable? What advantage does it give?

Python strings are immutable because their internal state cannot change after creation.

Advantages:

- Hashability: Strings can be used as dictionary keys and set elements.
- Safety in multithreading: No race conditions since the value cannot change.
- Memory optimization: Python interns small strings (reuses same object), saving memory.
- Security: Prevents accidental modification of important data.

```
Example: s = "hello"; s[0] = "H"  # Raises TypeError: str does not support item
assignment
```

Q2. Difference between deep copy and shallow copy.

Shallow Copy: Copies only the top-level object; nested objects are still shared by reference.

Deep Copy: Recursively copies all objects including nested ones; fully independent copy.

```
import copy
original = [[1, 2], [3, 4]]
shallow = copy.copy(original)   # Nested lists are shared
deep = copy.deepcopy(original) # Nested lists are independent
```

Modifying shallow[0][0] also affects original[0][0], but modifying deep[0][0] does not.

Q3. Explain memory management in Python.

- Python uses a private heap to store all objects and data structures.
- Reference counting tracks how many references point to an object; when count reaches 0, memory is freed.
- The Garbage Collector (gc module) handles cyclic references that reference counting cannot resolve.
- Memory allocator (pymalloc) manages small objects (< 512 bytes) efficiently using memory pools.
- The sys module provides tools like sys.getrefcount() to inspect reference counts.

Q4. How does Python handle dynamic typing internally?

Python variables are not typed — they are just names (references/pointers) bound to objects. Every object has a type stored internally in the PyObject struct (ob_type field).

When you write x = 10, Python creates an integer object in memory with type int and binds the name x to that object. When x = 'hello', x is rebound to a new string object. The old integer object is garbage collected if no references remain.

```
x = 10    # x points to int object
x = "hi" # x now points to str object; int object may be GC'd
```

Q5. Difference between 'is' and '==' with example.

== (Equality): Checks if values are equal (calls __eq__ method).

is (Identity): Checks if both variables point to the exact same object in memory.

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)  # True  (same values)
print(a is b)  # False (different objects in memory)
c = a
print(a is c)  # True  (same object)
```

Note: For small integers (-5 to 256) and interned strings, 'is' may return True due to Python's caching.

Q6. Why are sets faster than lists for searching?

Sets use a hash table internally. Searching in a set is O(1) average time because Python computes the hash of the value and directly locates the bucket.

Lists use linear search: O(n) time. Python must check each element one by one.

```
s = {1, 2, 3, ..., 1000000}
print(999999 in s)  # O(1) — instant
l = [1, 2, 3, ..., 1000000]
print(999999 in l)  # O(n) — scans up to 1 million elements
```

Q7. What happens internally when you run a Python program?

- Source code (.py) is read by the Python interpreter.
- Lexer tokenizes the source; Parser generates an Abstract Syntax Tree (AST).
- AST is compiled into bytecode (.pyc file stored in __pycache__).
- Python Virtual Machine (PVM) executes the bytecode instruction by instruction.
- The GIL (Global Interpreter Lock) ensures thread safety during execution.

Q8. Explain iterator vs iterable.

Iterable: Any object that implements __iter__() and can return an iterator. Examples: list, string, tuple, dict.

Iterator: An object that implements both __iter__() and __next__(). It remembers state and returns the next value on each call to next().

```
my_list = [1, 2, 3]  # iterable
it = iter(my_list)   # iterator
print(next(it))      # 1
print(next(it))      # 2
```

Q9. List comprehension vs normal loop — which is faster and why?

List comprehensions are faster than equivalent for-loops. They are optimized at the C level in CPython and avoid repeated attribute lookups (e.g., list.append). The bytecode generated is more compact.

```
# List comprehension (faster)
squares = [x*x for x in range(1000)]
# Normal loop (slower due to append overhead)
squares = []
for x in range(1000): squares.append(x*x)
```

Benchmark: comprehensions are typically 30–40% faster for simple operations.

Q10. Why tuples are used as dictionary keys but lists are not?

Dictionary keys must be hashable (immutable). Tuples are immutable, so their hash value never changes — they can safely be used as keys.

Lists are mutable. If you used a list as a key and then modified it, the hash would change, corrupting the dictionary structure. Python therefore raises TypeError if you try.

```
d = {}
d[(1, 2)] = "valid"     # Works — tuple is hashable
d[[1, 2]] = "invalid"   # TypeError: unhashable type: list
```

Unit 2: Functions, OOP & Exceptions

Q1. Difference between *args and **kwargs.

***args:** Collects extra positional arguments into a tuple.

****kwargs:** Collects extra keyword arguments into a dictionary.

```
def demo(*args, **kwargs):
    print(args)    # (1, 2, 3)
    print(kwargs)  # {"a": 10, "b": 20}
demo(1, 2, 3, a=10, b=20)
```

Q2. What are lambda functions and when should you avoid them?

Lambda functions are anonymous, single-expression functions defined with the lambda keyword.

```
square = lambda x: x * x  # equivalent to def square(x): return x*x
```

Avoid lambdas when:

- The logic is complex (more than one expression needed).
- The function needs a name for clarity or reuse.
- Debugging is important (lambdas show as <lambda> in tracebacks).
- Type hints or docstrings are required.

Q3. Explain closure in Python.

A closure is a function that remembers variables from its enclosing scope even after that scope has finished executing.

```
def outer(x):
    def inner(y):
        return x + y  # 'x' is from enclosing scope
    return inner
add5 = outer(5)
print(add5(3))  # 8 — x=5 is remembered
```

The free variable x is stored in inner.__closure__. Closures are the foundation of decorators and factory functions.

Q4. Difference between method overloading and method overriding in Python.

Method Overloading: Defining multiple methods with the same name but different parameters. Python does NOT natively support this — the last definition wins. It is simulated using default arguments or *args.

Method Overriding: A subclass provides its own implementation of a method defined in the parent class.

```
class Animal:
    def speak(self): return "..."
class Dog(Animal):
```

```
    def speak(self): return "Woof"  # Override
```

Q5. Explain Multiple Inheritance (Diamond Problem).

Multiple inheritance allows a class to inherit from more than one parent. The Diamond Problem occurs when two parents share a common ancestor, creating ambiguity about which version of a method to use.

```
class A:
    def hello(self): print('A')
class B(A): pass
class C(A): pass
class D(B, C): pass  # Diamond
D().hello()  # Which hello? B's, C's, or A's?
```

Python resolves this with MRO (Method Resolution Order) using the C3 linearization algorithm.

Q6. What is MRO (Method Resolution Order)?

MRO defines the order in which Python searches for methods in inheritance hierarchies. Python uses the C3 Linearization algorithm.

Rule: Left to right, depth first, but each class appears only once, and a class always appears before its parents.

```
class D(B, C): pass
print(D.__mro__)  # D -> B -> C -> A -> object
```

You can view it with ClassName.__mro__ or ClassName.mro(). The super() function follows MRO automatically.

Q7. Difference between class variable and instance variable.

Class Variable: Shared across all instances of the class. Defined directly in the class body.

Instance Variable: Unique to each instance. Defined inside __init__ using self.

```
class Dog:
    species = "Canine"      # class variable
    def __init__(self, name):
        self.name = name    # instance variable
d1 = Dog("Rex")
d2 = Dog("Buddy")
print(d1.species)  # Canine (shared)
print(d1.name)     # Rex (unique to d1)
```

Q8. Difference between @staticmethod, @classmethod, and instance method.

Instance method: Receives self (the instance) as first argument. Can access and modify instance and class state.

@classmethod: Receives cls (the class) as first argument. Can access/modify class state but not instance state. Used as factory methods.

@staticmethod: Receives neither self nor cls. Behaves like a regular function inside a class — used for utility functions logically related to the class.

```
class MyClass:
    count = 0
    def instance_method(self): return self
    @classmethod
    def class_method(cls): return cls.count
    @staticmethod
    def static_method(): return 'utility'
```

Q9. Difference between finally and else block.

finally: Always executes, regardless of whether an exception occurred. Used for cleanup (closing files, releasing resources).

else: Executes only if NO exception was raised in the try block.

```
try:
    result = 10 / 2
except ZeroDivisionError:
    print("Error")
else:
    print("Success:", result)  # runs if no exception
finally:
    print("Always runs")       # runs regardless
```

Q10. Why use 'with open()' instead of 'open()'?

The with statement (context manager) automatically closes the file when the block exits, even if an exception is raised. It calls __enter__() on entry and __exit__() on exit.

```
# Without with — risky
f = open("file.txt")
data = f.read()
f.close()  # may not run if an exception occurs
# With 'with' — safe and clean
with open("file.txt") as f:
    data = f.read()  # file is auto-closed after block
```

Unit 3: Modules, Packages & Database

Q1. What is __init__.py and why is it needed?

__init__.py marks a directory as a Python package, allowing it to be imported as a module. Without it, Python (in Python 2 and some Python 3 setups) won't recognize the directory as a package.

It can be empty or contain initialization code that runs when the package is imported. It also controls what is exported when someone uses from package import *.

Q2. How does Python find modules (Module Search Path)?

When you import a module, Python searches locations in this order:

- 1. The current directory (or script's directory).
- 2. PYTHONPATH environment variable directories (if set).
- 3. Installation-dependent default directories (site-packages, standard library).

These paths are stored in sys.path. You can inspect or modify it at runtime.

```
import sys
print(sys.path)  # shows all search locations
```

Q3. Difference between SQL and NoSQL databases practically.

SQL (e.g., MySQL, PostgreSQL): Structured tables with fixed schema, strong ACID transactions, uses SQL query language. Best for relational data (banking, ERP).

NoSQL (e.g., MongoDB, Redis): Schema-less, flexible documents/key-value/graph storage, horizontally scalable, eventual consistency. Best for unstructured or rapidly changing data (social media, IoT).

Q4. Why MongoDB is schema-less?

MongoDB stores data as BSON (Binary JSON) documents inside collections. Unlike SQL tables, collections do not enforce a fixed structure — each document can have different fields and data types.

This allows rapid iteration during development without running migration scripts. You can add/remove fields per document as requirements change.

Q5. Explain CRUD operations with real example.

- CREATE — Insert a new record.

```
db.users.insertOne({name: 'Alice', age: 25})
```

- READ — Query records.

```
db.users.find({age: {$gt: 18}})
```

- UPDATE — Modify existing records.

```
db.users.updateOne({name:'Alice'}, {$set:{age:26}})
```

- DELETE — Remove records.

```
db.users.deleteOne({name: 'Alice'})
```

Q6. What problems does Git solve?

- Version history: Tracks every change to code, enabling rollback to any previous state.
- Collaboration: Multiple developers can work on the same codebase simultaneously.
- Branching: Isolate features/bugs in separate branches without affecting main code.
- Merge conflicts: Provides tools to identify and resolve conflicting changes.
- Backup: Distributed copies on every developer's machine — no single point of failure.

Q7. Difference between git pull and git fetch.

git fetch: Downloads changes from the remote but does NOT merge them into your working branch. Safe — local branch is unchanged.

git pull: Fetches AND immediately merges the remote changes into your current branch. Equivalent to git fetch + git merge.

```
git fetch origin        # Download updates, inspect them
git merge origin/main   # Then manually merge
git pull origin main    # Fetch + merge in one step
```

Unit 4: Flask Framework

Q1. Why Flask is called a Micro Framework?

Flask is called a micro framework because it provides only the core essentials needed to build a web application — URL routing, request/response handling, and templating (via Jinja2). It does not bundle features like ORM, authentication, form validation, or admin panels.

This makes Flask lightweight, flexible, and easy to learn. Developers choose extensions (Flask-SQLAlchemy, Flask-Login) as needed, keeping the core minimal.

Q2. Explain Flask request lifecycle.

- 1. Client sends HTTP request to the Flask server.
- 2. Flask's WSGI app (Werkzeug) receives and parses the request.
- 3. URL routing maps the path to a view function using the @app.route decorator.
- 4. Before-request hooks run (e.g., authentication checks).
- 5. The view function executes, building a response.

- 6. After-request hooks run (e.g., adding headers).
- 7. Flask sends the HTTP response back to the client.

Q3. Difference between GET and POST in terms of security.

GET: Parameters are appended to the URL (visible in browser history, server logs, referrer headers). Not suitable for sensitive data (passwords, tokens).

POST: Data is sent in the request body, not in the URL. Not visible in browser history. More secure for sensitive data, though HTTPS is still required for true encryption in transit.

Q4. How sessions work in Flask?

Flask sessions store user data between requests. By default, session data is stored in a signed cookie on the client side using the SECRET_KEY for signing (not encryption).

```
from flask import session
app.secret_key = "your_secret"
@app.route('/login')
def login():
    session["user"] = "Alice"  # stored in cookie
```

Since data lives on the client, it is tamper-proof (signed) but not private. For private server-side sessions, use Flask-Session extension.

Q5. Difference between authentication and authorization.

Authentication: Verifies WHO you are. (Login with username/password, JWT token, OAuth).

Authorization: Determines WHAT you are allowed to do. (Role-based access, permissions, scope).

Example: Logging into Gmail is authentication. Determining whether you can access the Admin panel is authorization.

Q6. What happens when multiple users access Flask app simultaneously?

Flask's built-in development server is single-threaded and handles one request at a time. In production, Flask is deployed behind a WSGI server (Gunicorn, uWSGI) with multiple worker processes or threads.

Flask uses request context locals (werkzeug.local.LocalProxy) to isolate request and application contexts per thread/greenlet, ensuring that request object contains the correct data for each concurrent user.

Unit 5: Django Framework

Q1. Why Django follows MVT instead of MVC?

MVT (Model-View-Template) is Django's variation of MVC:

- Model: Same as MVC — handles database logic and data.
- View: Handles business logic and fetches data from models (equivalent to MVC Controller).
- Template: Handles presentation/HTML (equivalent to MVC View).

Django calls it MVT because Django itself acts as the Controller by routing URLs to views via urls.py, removing the need for a separate controller layer that developers must write.

Q2. Explain Django ORM internally.

Django ORM (Object Relational Mapper) maps Python classes (models) to database tables. Each model field (CharField, IntegerField) maps to a database column.

When you write User.objects.filter(age__gt=18), Django translates this to SQL: SELECT * FROM users WHERE age > 18. The queryset is lazy — SQL is only executed when the data is actually needed (iteration, list(), etc.).

Internally it uses a QuerySet API that builds SQL query objects, which are then executed by a database backend adapter (psycopg2 for PostgreSQL, etc.).

Q3. What happens when a request hits Django server?

- 1. WSGI server receives the HTTP request and hands it to Django.
- 2. Request passes through Middleware stack (security, session, auth checks).
- 3. URL dispatcher (urls.py) matches the path and calls the appropriate view function/class.
- 4. View processes business logic, queries the database via ORM if needed.
- 5. View renders a Template with context data, producing an HTML response.
- 6. Response passes back through Middleware.
- 7. HTTP response is returned to the client.

Q4. Why migrations are required in Django?

Migrations keep the database schema synchronized with your Django models. When you add, remove, or modify a field in a model, the database table must be updated to match.

Migrations create a version history of schema changes, allowing teams to apply changes consistently across development, staging, and production environments without manually writing ALTER TABLE SQL.

```
python manage.py makemigrations  # create migration file
python manage.py migrate         # apply changes to DB
```

Q5. Explain middleware in Django.

Middleware is a layer of hooks (plugins) that process every request before it reaches the view and every response before it is sent to the client. Middleware is defined as a list in settings.py (MIDDLEWARE).

- SecurityMiddleware: Enforces HTTPS, sets security headers.
- SessionMiddleware: Manages sessions.
- AuthenticationMiddleware: Associates users with requests.
- CommonMiddleware: Handles URL normalization.

Each middleware implements process_request() and/or process_response() methods.

Q6. How Django handles security (CSRF, XSS)?

CSRF (Cross-Site Request Forgery): Django generates a unique CSRF token for each session, embedded in forms via {% csrf_token %}. POST requests without the token are rejected (403).

XSS (Cross-Site Scripting): Django's template engine auto-escapes HTML by default, converting <, >, &, ' and " to safe HTML entities. Use {{ variable }} and it is escaped automatically.

Additional protections: SQL injection is prevented by the ORM using parameterized queries. Clickjacking is prevented by X-Frame-Options headers set by XFrameOptionsMiddleware.

Unit 6: RESTful APIs

Q1. Why REST APIs are stateless?

Stateless means each HTTP request from the client must contain all information needed to understand and process it. The server does not store any session information between requests.

Benefits: Scalability (any server in a cluster can handle any request), simplicity (no server-side session storage), reliability (no state corruption across requests). Authentication is sent with every request (e.g., via JWT token or API key in headers).

Q2. Why PUT is idempotent but POST is not?

PUT: Replaces the entire resource at a specific URL. Calling it multiple times with the same data produces the same result — resource stays the same.

POST: Creates a new resource. Each call creates a NEW resource with a new ID, so multiple calls produce multiple new records.

```
PUT /users/1  {name: 'Alice'}  // Always results in user 1 having name Alice
POST /users   {name: 'Alice'}  // Creates a new user each time
```

Q3. Explain idempotency with example.

An operation is idempotent if performing it multiple times produces the same result as performing it once.

- Idempotent HTTP methods: GET, PUT, DELETE, HEAD.
- Non-idempotent: POST (creates new resource each time).

Example: DELETE /orders/5 — deletes order 5. Calling it again returns 404 (order already gone), but the final state of the system is the same: order 5 does not exist.

Q4. What happens inside HTTP request-response cycle?

- 1. Client resolves domain to IP via DNS.
- 2. TCP connection is established (3-way handshake).
- 3. Client sends HTTP request (method, headers, body).
- 4. Server processes request, executes application logic.
- 5. Server sends HTTP response (status code, headers, body).
- 6. Client (browser/app) processes the response.
- 7. TCP connection is closed (or kept alive via Connection: keep-alive).

Q5. Why JSON is preferred over XML?

- JSON is lighter — less verbose, no opening/closing tags.
- JSON is natively supported in JavaScript (JSON.parse / JSON.stringify).
- JSON maps directly to Python dicts, lists, and primitive types.
- JSON parsing is faster and consumes less bandwidth.
- XML is better for document-centric data requiring namespaces and schemas (SOAP, legacy systems).

Q6. Difference between 401 and 403 status code.

401 Unauthorized: The client is NOT authenticated. No valid credentials provided. You need to log in first.

403 Forbidden: The client IS authenticated but does NOT have permission to access the resource. You are logged in, but you lack the required role or privilege.

Q7. What is API versioning and why needed?

API versioning allows you to release new, breaking changes to an API while keeping old versions working for existing clients. This prevents breaking changes from disrupting clients that depend on the old API.

Approaches: URL versioning (/api/v1/, /api/v2/), Header versioning (Accept: application/vnd.api+json;version=2), or Query parameter (?version=2).

Q8. How do you secure REST APIs?

- Authentication: Use JWT tokens, OAuth 2.0, or API keys in headers.
- HTTPS: Always encrypt traffic using TLS/SSL.
- Rate limiting: Prevent abuse/DDoS by limiting requests per client.
- Input validation: Sanitize all inputs to prevent injection attacks.
- CORS: Configure allowed origins to prevent unauthorized cross-origin requests.
- Least privilege: Return only the data the client needs (no over-fetching sensitive fields).

Top 10 Examiner Favourite Questions — Quick Reference

Q1. MRO in Python

Method Resolution Order: defines method lookup sequence in inheritance. Python uses C3 Linearization. Order: left to right, depth first. Use ClassName.__mro__ to inspect.

Q2. 'is' vs '==' comparison

== checks value equality; 'is' checks identity (same memory object). a = [1,2]; b = [1,2]: a==b is True, a is b is False.

Q3. Deep copy vs shallow copy

Shallow copy: top-level new object, nested objects shared. Deep copy: completely independent recursive copy. Use copy.copy() and copy.deepcopy() respectively.

Q4. Django request flow

Request → Middleware → URL dispatcher → View → ORM/Template → Middleware → Response. Django acts as the controller routing through urls.py.

Q5. Flask vs Django difference

Flask: Micro, minimalist, unopinionated, choose your own extensions, great for APIs and small apps.
Django: Full-stack, batteries-included (ORM, Admin, Auth, Migrations), opinionated, best for large structured applications.

Q6. PUT vs POST (Idempotency)

PUT is idempotent: replaces a specific resource — same result every call. POST is not idempotent: creates a new resource on every call.

Q7. Class vs Instance variables

Class variables are shared across all instances (defined at class level). Instance variables are unique per object (defined via self in __init__).

Q8. @staticmethod vs @classmethod

@classmethod receives cls as first argument and can access class state; used as factory methods. @staticmethod receives nothing (no self/cls) and is a pure utility function placed inside the class for organization.

Q9. Django ORM concept

Maps Python model classes to database tables. QuerySets are lazy (SQL runs only when data is evaluated). Methods like filter(), exclude(), get() are translated to parameterized SQL, preventing SQL injection.

Q10. REST API stateless nature

Each request must contain all context (auth token, params). Server stores no client state between requests. Enables horizontal scaling — any server can handle any request.

End of Document — Best of luck for your Viva!